
AUDITORIA COMPLETA

PRAGA Living Platform

Next.js 16 + Supabase + NextAuth + Framer Motion

Fecha: 7 de junio de 2026

Resumen de Hallazgos

4 Criticos | 8 Altos | 12 Medios | 6 Bajos | 5 Informativos

1. Resumen Ejecutivo

PRAGA Living es una plataforma inmobiliaria premium construida con Next.js 16.1, React 19, TypeScript, TailwindCSS 4, Framer Motion, Supabase, NextAuth, y un stack de 40+ dependencias. La aplicacion funciona como un sitio single-page con 15 secciones animadas, un panel administrativo completo, y un chatbot de IA. Esta auditoria revisa la totalidad del codigo fuente, identificando bugs criticos, vulnerabilidades de seguridad, problemas de rendimiento, y oportunidades de mejora en la experiencia de usuario y la mantenibilidad del codigo.

Los hallazgos mas criticos incluyen: contraseñas almacenadas en texto plano sin hash, credenciales hardcodeadas accesibles en el cliente, ausencia de proteccion CSRF en todas las rutas API de escritura, imagenes duplicadas en amenidades, uso de styled-jsx incompatible con App Router, y falta de optimizacion de imagenes en varios componentes. A continuacion se detallan todos los hallazgos organizados por categoria y severidad.

1.1 Stack Tecnologico

Item	Detail
Framework	Next.js 16.1.1 (App Router, standalone output)
UI	React 19, TailwindCSS 4, shadcn/ui, Framer Motion
Base de Datos	Supabase (PostgreSQL) + Prisma (SQLite fallback) + Hardcoded
Autenticacion	NextAuth v4 (credentials provider, JWT)
3D	Three.js / React Three Fiber / Drei
Mapas	Leaflet / React-Leaflet
Email	Resend (free tier)
IA	z-ai-web-dev-sdk (chat completions)
Animacion	Framer Motion + Lenis smooth scroll
PDF	jsPDF (cotizaciones)
Estado	Zustand + React Query implicito (fetch manual)

2. Auditoria de Seguridad

Severit y	Component	Issue
--------------	-----------	-------

CRITICAL	auth-config.ts	Contraseñas almacenadas en texto plano. admin.password === credentials.password compara plaintext directamente. No usa bcrypt/argon2. La tabla admin_users en Supabase almacena contraseñas sin hash.
CRITICAL	data.ts	Credenciales admin hardcodeadas: admin/praga2024. Estas credenciales están embebidas en el bundle del cliente y son accesibles desde el navegador. Cualquier usuario puede inspeccionar el código y obtener acceso.
CRITICAL	API routes	Sin protección CSRF. Todas las rutas POST/PUT (/api/leads, /api/apartments, /api/site-config, /api/floor-plans) no verifican el origen de la petición. Un sitio malicioso puede enviar peticiones en nombre de un usuario autenticado.
HIGH	API routes	Sin autenticación en API de escritura. Las rutas PUT de /api/apartments, /api/leads, /api/amenities y POST de /api/site-config no validan session. Cualquier visitante anónimo puede cambiar precios, estados y configuración.
HIGH	supabase.ts	createAdminSupabaseClient() usa la service role key que está expuesta como variable de entorno del servidor, pero el fallback a anon key cuando service role no está disponible debilita la seguridad de escritura.
HIGH	middleware.ts	El middleware solo protege /admin/* pero las rutas API (/api/apartments, /api/leads, etc.) quedan completamente desprotegidas. Un atacante puede manipular datos directamente via curl.
MEDIUM	email.ts	El remitente de emails usa onboarding@resend.dev (dominio genérico). El link en el email apunta a localhost:3000/admin en vez de la URL de producción.
MEDIUM	ChatIA.tsx	El chatbot envía mensajes de usuario directamente al endpoint /api/chat sin sanitización. Podría inyectarse contenido en el prompt del sistema.
LOW	next.config.ts	ignoreBuildErrors: true y reactStrictMode: false. Esto oculta errores de TypeScript en build y desactiva la detección de efectos duplicados en desarrollo.

3. Bugs Funcionales

Severit y	Component	Issue
CRITICAL	Amenidades.tsx	Imágenes duplicadas: Sauna y Turco usan /images/renders/vitality-pool.png (misma imagen). Ludoteca usa /images/renders/coworking.png. Los comentarios TODO lo confirman. El usuario ve imágenes repetidas que no corresponden a la amenidad.
HIGH	PlantaInteractiva.tsx	Uso de styled-jsx (<style jsx global>) que causa advertencias de hydration en Next.js App Router. styled-jsx es un feature de Pages Router y no está soportado oficialmente en App Router. Puede causar FOUC (Flash of Unstyled Content).
HIGH	Ubicacion.tsx	Mismo problema de styled-jsx global para estilos de Leaflet. Dos bloques <style jsx global> en el mismo componente con más de 150 líneas de CSS inyectado globalmente.

HIGH	data.ts	Inconsistencia en nombres de amenidades: data.ts tiene "Sauna" y "Bano Turco" como amenidades separadas, pero Amenidades.tsx las combina como "Sauna & Turco". La data fallback y el componente usan esquemas diferentes.
HIGH	PlantaInteractiva.tsx	El tooltip usa posicion fixed con clientX/clientY pero no tiene boundary checking. En mobile o con scroll, el tooltip aparece fuera del viewport y no es visible.
MEDIUM	Hero.tsx	El evento hero-ready se dispara con window.dispatchEvent que no es confiable. Si la imagen se carga desde cache, el evento onload no se dispara y la pantalla de carga permanece indefinidamente hasta que pasen los 2200ms.
MEDIUM	Tipologias.tsx	Las tipologias en defaultTypologies (Tipo A 75m2, Tipo B 48m2, Tipo A Premium 78m2) no coinciden con las tipologias de data.ts (Tipo A 74.75m2, Tipo B 57m2, Tipo A+ 97.45m2). Las areas y nombres son inconsistentes.
MEDIUM	data.ts	checkSupabase() cachea el resultado en supabaseChecked sin expiracion. Si Supabase se cae durante la ejecucion, la app seguira intentando usar Supabase hasta que se reinicie el servidor.
MEDIUM	AdminPanel.tsx	El componente tiene 1000+ lineas con ~25 estados useState. La logica de UI y datos esta completamente acoplada. Dificil de mantener y propenso a bugs de estado.
MEDIUM	page.tsx	El loader depende de un evento custom (hero-ready) que se dispara solo para la primera imagen. Si esa imagen falla, la pagina queda bloqueada en el loader hasta los 2200ms de timeout.
LOW	Galeria.tsx	Las imagenes usan en vez de <Image> de Next.js. No hay optimizacion automatica de imagenes, lazy loading nativo, ni formato WebP/AVIF.
LOW	Navigation.tsx	Los hamburger menu items usan el mismo color para scrolled y no-scrolled (bg-[#F5F1EA] en ambos casos), haciendo el codigo redundante.

4. Rendimiento

Severit y	Component	Issue
HIGH	page.tsx	La pagina principal importa 15+ componentes pesados sincronicamente (incluyendo BuildingScene con Three.js, MapView con Leaflet, ExplorarEdificio). No hay lazy loading para componentes que estan abajo del fold.
HIGH	Amenidades.tsx	Las 6 imagenes de amenidades se cargan con priority solo en selected===0. Las demas se cargan solo cuando el usuario interactua, causando delay visible en la transicion.
MEDIUM	useSiteConfig.ts	El hook hace un fetch a /api/site-config en CADA montaje de componente. Como 10+ componentes lo usan, se generan 10+ peticiones HTTP identicas en el primer render. No hay cache ni deduplicacion.
MEDIUM	data.ts	generateFallbackApartments() se ejecuta en cada llamada a getApartments() cuando no hay Supabase/Prisma, generando 110 objetos en memoria cada vez. Deberia cachearse.

MEDIUM	PlantaInteractiva.tsx	El SVG overlay usa preserveAspectRatio="none" que distorsiona los poligonos cuando la imagen no mantiene la proporcion exacta. Los hotspots pueden no alinearse con los apartamentos reales.
MEDIUM	Navigation.tsx	El scroll handler se ejecuta en cada frame sin throttle/debounce. Itera sobre 12 secciones del DOM en cada evento de scroll, verificando getBoundingClientRect().
LOW	AdminPanel.tsx	fetchData() hace 3 peticiones API secuenciales (apartments, leads, amenities) cuando podria hacerlas en paralelo con Promise.all (ya lo hace, pero la logica de re-fetch no es optima).
LOW	globals.css	Las animaciones lightStreakMove son infinitas y corren en segundo plano incluso cuando el hero no es visible. Deberian pausarse con IntersectionObserver.

5. UX y Accesibilidad

Severit y	Component	Issue
HIGH	PlantaInteractiva.tsx	En mobile, la disposicion de 3 columnas (selector + plano + detalle) se aplasta. El selector de pisos tiene max-height de 580px que no funciona bien en pantallas pequenas. El panel de detalle no es colapsable.
HIGH	Hero.tsx	Los tiempos de animacion son excesivamente largos: 2.5s para el subtítulo, 2.8s para el título, 3.4s para "Living", 3.8s para tagline, 4.2s para CTAs, 5s para indicador, 5.5s para scroll indicator. Total: 5.5 segundos antes de que el usuario pueda interactuar.
MEDIUM	ChatIA.tsx	El chat tiene ancho fijo de 360px y no es responsive. En pantallas menores a 400px, el chat se desborda del viewport. Los mensajes no tienen max-width relativo al contenedor.
MEDIUM	Navigation.tsx	El menu mobile muestra los 12 items en vertical sin scroll. En pantallas cortas, los items inferiores no son accesibles sin scroll dentro del overlay.
MEDIUM	Contacto.tsx	La validacion de telefono colombiano es muy estricta. El regex requiere formato exacto y rechaza numeros validos con diferentes formatos de separacion.
MEDIUM	Global	No hay estados de error visibles para los componentes cuando falla la carga de datos (useSiteConfig). Los componentes simplemente muestran los defaults sin indicar al usuario que algo fallo.
LOW	WhatsAppButton.tsx	El boton tiene animacion de ping infinita (animate-ping opacity-20) que puede ser molesta y consumir recursos GPU innecesariamente.
LOW	Footer.tsx	Los links legales (Politica de Privacidad, Terminos de Uso) apuntan a "#" sin paginas reales. Son links vacios.

6. Calidad deCodigo y Mantenibilidad

Severit y	Component	Issue
HIGH	AdminPanel.tsx	Componente monolitico de 1000+ lineas con 25+ estados. Deberia dividirse en subcomponentes (DashboardTab, ApartmentsTab, LeadsTab, etc.) usando composicion en vez de un solo archivo.
HIGH	data.ts	Triple fallback (Supabase -> Prisma -> Hardcoded) con verificacion sincrona de disponibilidad en cada llamada. El patron esta repetido 8+ veces con ligeras variaciones. Deberia abstraerse en un patron generico.
MEDIU M	Multiple	Tipado inconsistente: muchos componentes usan any implicito o type assertions (as any). ApartmentZone, UnitData, FloorConfig en PlantaInteractiva.tsx duplican tipos que ya existen en data.ts.
MEDIU M	API routes	Todos los catch blocks estan vacios (// silently fail, // fall through). Los errores se ignoran completamente sin logging, haciendo imposible diagnosticar problemas en produccion.
MEDIU M	data.ts	El patron de checkSupabase() + checkDb() se ejecuta en CADA llamada a funciones de datos. No hay un mecanismo de circuit breaker ni reintento con backoff.
MEDIU M	quotes-store.ts	El almacen de cotizaciones usa globalThis para persistir en memoria. Se pierde al reiniciar el servidor y no escala en serverless (cada instancia tiene su propio globalThis).
LOW	Ubicacion.tsx	Los datos de POI (locationLayers) estan hardcodedos en el componente en vez de venir del CMS (site-config). No son editables desde el panel admin.
LOW	globals.css	Clases CSS duplicadas: .font-cormorant y .font-inter se definen en globals.css pero los componentes usan font-[family-name:var(--font-cormorant)] inline. Dos mecanismos diferentes para lo mismo.

7. Configuracion y Despliegue

Severit y	Component	Issue
HIGH	next.config.ts	ignoreBuildErrors: true permite que errores de TypeScript pasen desapercibidos en produccion. reactStrictMode: false desactiva la deteccion de bugs de efectos en desarrollo.
MEDIU M	env vars	Variables de entorno criticas (NEXT_PUBLIC_SUPABASE_ANON_KEY, NEXTAUTH_SECRET) estan expuestas en el bundle del cliente. SUPABASE_SERVICE_ROLE_KEY deberia usarse SOLO en el servidor.
MEDIU M	email.ts	Resend free tier solo envia a yecos11@gmail.com. El dominio onboarding@resend.dev no es profesional. Se necesita verificar pragaliving.com en Resend para produccion.

LOW	prisma/schema	El schema de Prisma define 4 modelos (Apartment, Amenity, Lead, AdminUser) pero en produccion se usa Supabase. Prisma es solo fallback local, creando duplicacion de schema.
LOW	tailwind.config.ts	El content array incluye ./pages/** y ./components/** que son rutas del Pages Router. En App Router solo necesita ./src/app/** y ./src/components/**.
INFO	Google Analytics	GoogleAnalytics component existe pero no tiene Measurement ID configurado. El componente se monta pero no envia datos.

8. Problemas Arquitectonicos

Severit y	Component	Issue
HIGH	Data Layer	Triple fallback (Supabase -> Prisma -> Hardcoded) introduce complejidad excesiva. Los 3 origenes de datos pueden estar dessincronizados. Las escrituras van a Supabase pero las lecturas pueden venir de cualquier fuente.
MEDIU M	State Management	No hay capa de cache/deduplicacion. useSiteConfig hace fetch en cada montaje. Los datos de apartamentos se cargan con fetch directo sin SWR/React Query. Resultado: peticiones redundantes y datos potencialmente stale.
MEDIU M	CMS Architecture	site-config.json tiene 579 lineas con toda la configuracion. El editor admin (SiteConfigEditor) permite editar secciones individuales, pero la carga inicial trae todo. Deberia usar lazy loading por seccion.
MEDIU M	Component Coupling	PlantaInteractiva hace fetch directo a /api/floor-plans en vez de usar useSiteConfig o un hook dedicado. Otros componentes usan useSiteConfig. No hay patron consistente para acceso a datos.
LOW	3D Module	BuildingScene.tsx importa Three.js y React Three Fiber pero no se usa lazy loading. Estos modulos pesan ~500KB y se cargan incluso si el usuario nunca scrollea al section ExplorarEdificio.
INFO	WebSocket	El directorio examples/websocket/ contiene implementaciones de referencia que no estan integradas en la app. Deberia limpiarse o integrarse.

9. Plan de Accion Priorizado

9.1 Fase 1 - Critico (Semana 1)

P1-1: Hash de contraseñas

Implementar bcrypt en auth-config.ts y data.ts. Migrar admin_users en Supabase para usar contraseñas hasheadas. Eliminar credenciales hardcodeadas del bundle del cliente.

P1-2: Proteccion API

Agregar verificacion de session en todas las rutas API de escritura (PUT/POST). Implementar tokens CSRF o verificar el header Origin.

P1-3: Imagenes duplicadas

Reemplazar las imagenes placeholder de Sauna, Turco y Ludoteca en Amenidades.tsx y data.ts con imagenes dedicadas o placeholders visuales que indiquen "Proximamente".

9.2 Fase 2 - Alto (Semana 2)

P2-1: Eliminar styled-jsx

Reemplazar los 3 bloques `<style jsx global>` (PlantaInteractiva, Ubicacion x2) con CSS modules o CSS en globals.css. Esto elimina las advertencias de hydration y FOUC.

P2-2: Lazy loading

Importar con `dynamic()` los componentes pesados: `ExplorarEdificio` (Three.js), `MapView` (Leaflet), `Galeria`, `ChatIA`. Reducir el bundle inicial en ~1MB.

P2-3: Cache de datos

Implementar SWR o React Query en `useSiteConfig`. Agregar deduplicacion de peticiones con cache en memoria para datos que no cambian frecuentemente.

P2-4: Autenticacion API

Crear un helper `withAuth()` que envuelva los handlers de API routes, verificando la session antes de ejecutar la logica de negocio.

9.3 Fase 3 - Medio (Semanas 3-4)

P3-1: Refactor AdminPanel

Dividir `AdminPanel.tsx` en subcomponentes independientes por tab. Mover logica de datos a custom hooks. Reducir de 1000+ a ~200 lineas por archivo.

P3-2: Unificar tipologias

Alinear las tipologias en `Tipologias.tsx`, `data.ts`, y `PlantaInteractiva.tsx` para que usen los mismos nombres, areas y descripciones. Centralizar en `site-config`.

P3-3: Mejora UX Hero

Reducir tiempos de animacion de 5.5s a 3s maximo. Hacer el CTA visible antes (delay 1.5s en vez de 4.2s). El usuario debe poder interactuar en menos de 2 segundos.

P3-4: Manejo de errores

Reemplazar catch blocks vacios con logging estructurado (console.error con contexto). Agregar estados de error visibles en componentes cuando falla la carga de datos.

P3-5: Responsive Planta

Rediseñar PlantaInteractiva para mobile: colapsar selector y detalle en sheet/drawer, mostrar plano a pantalla completa con tap para seleccionar.

9.4 Fase 4 - Bajo (Mes 2)

P4-1: Optimizacion Next/Image

Migrar todos los a <Image> de Next.js en Galeria, Tipologias, Footer y Navigation. Agregar sizes y priority apropiados.

P4-2: Config next.config

Activar reactStrictMode y remover ignoreBuildErrors. Corregir errores de TypeScript que emerjan.

P4-3: Animaciones bajo demanda

Pausar animaciones CSS infinitas (lightStreakMove) cuando el hero no es visible usando IntersectionObserver.

P4-4: Limpieza de codigo

Eliminar examples/websocket, mover POI data a site-config, unificar mecanismos de fuentes CSS, agregar paginas legales reales.

10. Inventario de Archivos Auditados

La auditoria cubre la totalidad de los archivos fuente del proyecto. A continuacion se listan los archivos principales revisados con su funcion y estado de revision.

Archivo	Rol	Lineas	Estado
src/app/page.tsx	Pagina principal (SPA)	132	Revisado
src/app/layout.tsx	Layout raiz + SEO	91	Revisado
src/lib/data.ts	Capa de datos central	782	Revisado
src/lib/supabase.ts	Cliente Supabase	31	Revisado
src/lib/auth-config.ts	Config NextAuth	97	Revisado
src/lib/email.ts	Envio de emails	70	Revisado
src/hooks/useSiteConfig.ts	Hook configuracion	22	Revisado

src/components/praga/Hero.tsx	Seccion Hero	269	Revisado
src/components/praga/Amenidades.tsx	Seccion Amenidades	257	Revisado
src/components/praga/PlantaInteractiva.tsx	Planta interactiva	681	Revisado
src/components/praga/Navigation.tsx	Navegacion	179	Revisado
src/components/praga/Contacto.tsx	Formulario contacto	343	Revisado
src/components/praga/Tipologias.tsx	Tipologias	251	Revisado
src/components/praga/Ubicacion.tsx	Mapa + POIs	469	Revisado
src/components/praga/Galeria.tsx	Galeria fotos	165	Revisado
src/components/praga/Inversion.tsx	Indicadores	142	Revisado
src/components/praga/Footer.tsx	Pie de pagina	107	Revisado
src/components/praga/ChatIA.tsx	Chatbot IA	199	Revisado
src/components/praga/WhatsAppButton.tsx	Boton WhatsApp	44	Revisado
src/components/praga/AdminPanel.tsx	Panel admin	1000+	Revisado
src/middleware.ts	Proteccion rutas	11	Revisado
src/app/api/leads/route.ts	API leads	83	Revisado
src/app/api/site-config/route.ts	API configuracion	54	Revisado
src/app/api/floor-plans/route.ts	API plantas	47	Revisado
src/app/api/chat/route.ts	API chat IA	49	Revisado
src/app/api/apartments/route.ts	API apartamentos	36	Revisado
src/app/globals.css	Estilos globales	188	Revisado
next.config.ts	Config Next.js	12	Revisado
package.json	Dependencias	114	Revisado
tailwind.config.ts	Config Tailwind	64	Revisado